

Industrial Protocols Lab Week #3

AXI Register-Mapped Peripheral

Introduction

The Advanced extensible Interface (AXI) is a high-performance bus protocol developed by ARM as part of the AMBA specification. It supports independent read and write channels, making data transfers efficient and suitable for modern System-on-Chip (SoC) designs. In this lab, an AXI register-mapped peripheral is implemented and verified using a Verilog testbench.

Register-Mapped Peripheral

A register-mapped peripheral is a hardware module whose internal registers are assigned unique memory addresses. The processor communicates with the peripheral by reading from or writing to these addresses, allowing configuration, status monitoring, and data transfer through standard memory access operations.

Design Implementation

The AXI register-mapped peripheral is implemented using the following steps:

1. Define the AXI interface signals for write and read transactions.
2. Create internal registers to store configuration and data values.
3. Implement the AXI write channel to receive address and write data from the master.
4. Update the selected register when a valid write transaction occurs.
5. Implement the AXI read channel to return the requested register contents.
6. Generate the required AXI handshake signals (`READY`, `VALID`, and response signals).
7. Reset all registers and control signals when the reset signal is asserted.
8. Verify proper communication between the AXI master and the peripheral.

Design Code:

```
`timescale 1ns / 1ps

module axi4_lite_master #(
    parameter ADDR_WIDTH = 32,
    parameter DATA_WIDTH = 32
)
```

```

(
    input wire ACLK,
    input wire ARESETN,
    input wire START_READ,
    input wire START_WRITE,
    input wire [ADDR_WIDTH-1:0] address,
    input wire [DATA_WIDTH-1:0] W_data,
    output wire [DATA_WIDTH-1:0] R_data,
    output wire READ_COMPLETE,
    output wire WRITE_COMPLETE,
    input wire M_ARREADY,
    output reg [ADDR_WIDTH-1:0] M_ARADDR,
    output reg M_ARVALID,
    input wire [DATA_WIDTH-1:0] M_RDATA,
    input wire [1:0] M_RRESP,
    input wire M_RVALID,
    output reg M_RREADY,
    input wire M_AWREADY,
    output reg [ADDR_WIDTH-1:0] M_AWADDR,
    output reg M_AWVALID,
    input wire M_WREADY,
    output reg [DATA_WIDTH-1:0] M_WDATA,
    output reg [3:0] M_WSTRB,
    output reg M_WVALID,
    input wire [1:0] M_BRESP,
    input wire M_BVALID,
    output reg M_BREADY
);

localparam IDLE = 3'b000;
localparam WRITE_ADDR = 3'b001;
localparam WRITE_DATA = 3'b010;
localparam WRITE_RESP = 3'b011;
localparam READ_ADDR = 3'b100;
localparam READ_DATA = 3'b101;

reg [2:0] state, next_state;
reg read_start_reg, write_start_reg;
reg [DATA_WIDTH-1:0] rdata_reg;

assign R_data = rdata_reg;
assign READ_COMPLETE = (state == READ_DATA) && M_RVALID && M_RREADY;

```

```
assign WRITE_COMPLETE = (state == WRITE_RESP) && M_BVALID && M_BREADY;
```

```
always @(posedge ACLK or negedge ARESETN) begin
    if (!ARESETN) begin
        state <= IDLE;
    end else begin
        state <= next_state;
    end
end
```

```
always @(posedge ACLK or negedge ARESETN) begin
    if (!ARESETN) begin
        read_start_reg <= 1'b0;
        write_start_reg <= 1'b0;
    end else begin
        read_start_reg <= START_READ;
        write_start_reg <= START_WRITE;
    end
end
```

```
always @(posedge ACLK or negedge ARESETN) begin
    if (!ARESETN) begin
        rdata_reg <= {DATA_WIDTH{1'b0}};
    end else if (state == READ_DATA && M_RVALID && M_RREADY) begin
        rdata_reg <= M_RDATA;
    end
end
```

```
always @(*) begin
    next_state = state;
    case (state)
        IDLE: begin
            if (write_start_reg) begin
                next_state = WRITE_ADDR;
            end else if (read_start_reg) begin
                next_state = READ_ADDR;
            end else begin
                next_state = IDLE;
            end
        end
        WRITE_ADDR: begin
            if (M_AWVALID && M_AWREADY) begin
```

```

        next_state = WRITE_DATA;
    end else begin
        next_state = WRITE_ADDR;
    end
end
WRITE_DATA: begin
    if (M_WVALID && M_WREADY) begin
        next_state = WRITE_RESP;
    end else begin
        next_state = WRITE_DATA;
    end
end
WRITE_RESP: begin
    if (M_BVALID && M_BREADY) begin
        next_state = IDLE;
    end else begin
        next_state = WRITE_RESP;
    end
end
READ_ADDR: begin
    if (M_ARVALID && M_ARREADY) begin
        next_state = READ_DATA;
    end else begin
        next_state = READ_ADDR;
    end
end
READ_DATA: begin
    if (M_RVALID && M_RREADY) begin
        next_state = IDLE;
    end else begin
        next_state = READ_DATA;
    end
end
default: next_state = IDLE;
endcase
end

always @(*) begin
    M_ARADDR = {ADDR_WIDTH{1'b0}};
    M_ARVALID = 1'b0;
    M_RREADY = 1'b0;
    M_AWADDR = {ADDR_WIDTH{1'b0}};

```

```

M_AWVALID = 1'b0;
M_WDATA = {DATA_WIDTH{1'b0}};
M_WSTRB = 4'b0000;
M_WVALID = 1'b0;
M_BREADY = 1'b0;

case (state)
    WRITE_ADDR: begin
        M_AWADDR = address;
        M_AWVALID = 1'b1;
        M_BREADY = 1'b1;
    end
    WRITE_DATA: begin
        M_WDATA = W_data;
        M_WSTRB = 4'b1111;
        M_WVALID = 1'b1;
        M_BREADY = 1'b1;
    end
    WRITE_RESP: begin
        M_BREADY = 1'b1;
    end
    READ_ADDR: begin
        M_ARADDR = address;
        M_ARVALID = 1'b1;
        M_RREADY = 1'b1;
    end
    READ_DATA: begin
        M_RREADY = 1'b1;
    end
    default: begin
    end
endcase
end

endmodule

module axi4_lite_slave #(
    parameter ADDR_WIDTH = 32,
    parameter DATA_WIDTH = 32,
    parameter NUM_REGISTERS = 32
)

```

```

(
    input wire ACLK,
    input wire ARESETN,
    input wire [ADDR_WIDTH-1:0] S_ARADDR,
    input wire S_ARVALID,
    output reg S_ARREADY,
    input wire S_RREADY,
    output reg [DATA_WIDTH-1:0] S_RDATA,
    output reg [1:0] S_RRESP,
    output reg S_RVALID,
    input wire [ADDR_WIDTH-1:0] S_AWADDR,
    input wire S_AWVALID,
    output reg S_AWREADY,
    input wire [DATA_WIDTH-1:0] S_WDATA,
    input wire [3:0] S_WSTRB,
    input wire S_WVALID,
    output reg S_WREADY,
    input wire S_BREADY,
    output reg [1:0] S_BRESP,
    output reg S_BVALID
);

localparam IDLE = 3'b000;
localparam WRITE_ADDR = 3'b001;
localparam WRITE_DATA = 3'b010;
localparam WRITE_RESP = 3'b011;
localparam READ_ADDR = 3'b100;
localparam READ_DATA = 3'b101;

reg [2:0] state, next_state;
reg [ADDR_WIDTH-1:0] addr_reg;
reg [DATA_WIDTH-1:0] registers [0:NUM_REGISTERS-1];

wire write_addr_handshake;
wire write_data_handshake;
wire read_addr_handshake;
wire read_data_handshake;

assign write_addr_handshake = S_AWVALID && S_AWREADY;
assign write_data_handshake = S_WVALID && S_WREADY;
assign read_addr_handshake = S_ARVALID && S_ARREADY;
assign read_data_handshake = S_RVALID && S_RREADY;

```

```

always @(posedge ACLK or negedge ARESETN) begin
    if (!ARESETN) begin
        state <= IDLE;
    end else begin
        state <= next_state;
    end
end

always @(posedge ACLK or negedge ARESETN) begin
    if (!ARESETN) begin
        addr_reg <= {ADDR_WIDTH{1'b0}};
    end else if (state == READ_ADDR && read_addr_handshake) begin
        addr_reg <= S_ARADDR;
    end
end

integer i;
always @(posedge ACLK or negedge ARESETN) begin
    if (!ARESETN) begin
        for (i = 0; i < NUM_REGISTERS; i = i + 1) begin
            registers[i] <= {DATA_WIDTH{1'b0}};
        end
    end else begin
        if (state == WRITE_DATA && write_data_handshake) begin
            if (S_WSTRB[0]) registers[S_AWADDR][7:0] <= S_WDATA[7:0];
            if (S_WSTRB[1]) registers[S_AWADDR][15:8] <= S_WDATA[15:8];
            if (S_WSTRB[2]) registers[S_AWADDR][23:16] <= S_WDATA[23:16];
            if (S_WSTRB[3]) registers[S_AWADDR][31:24] <= S_WDATA[31:24];
        end
    end
end

always @(*) begin
    next_state = state;
    case (state)
        IDLE: begin
            if (S_AWVALID) begin
                next_state = WRITE_ADDR;
            end else if (S_ARVALID) begin
                next_state = READ_ADDR;
            end else begin

```

```

        next_state = IDLE;
    end
end
WRITE_ADDR: begin
    if (write_addr_handshake) begin
        next_state = WRITE_DATA;
    end else begin
        next_state = WRITE_ADDR;
    end
end
WRITE_DATA: begin
    if (write_data_handshake) begin
        next_state = WRITE_RESP;
    end else begin
        next_state = WRITE_DATA;
    end
end
WRITE_RESP: begin
    if (S_BVALID && S_BREADY) begin
        next_state = IDLE;
    end else begin
        next_state = WRITE_RESP;
    end
end
READ_ADDR: begin
    if (read_addr_handshake) begin
        next_state = READ_DATA;
    end else begin
        next_state = READ_ADDR;
    end
end
READ_DATA: begin
    if (read_data_handshake) begin
        next_state = IDLE;
    end else begin
        next_state = READ_DATA;
    end
end
default: next_state = IDLE;
endcase
end

```



```

always @(*) begin
    S_ARREADY = 1'b0;
    S_RDATA = {DATA_WIDTH{1'b0}};
    S_RRESP = 2'b00;
    S_RVALID = 1'b0;
    S_AWREADY = 1'b0;
    S_WREADY = 1'b0;
    S_BRESP = 2'b00;
    S_BVALID = 1'b0;

    case (state)
        WRITE_ADDR: begin
            S_AWREADY = 1'b1;
            S_WREADY = 1'b1;
        end
        WRITE_DATA: begin
            S_WREADY = 1'b1;
            S_BVALID = 1'b1;
            S_BRESP = 2'b00;
        end
        WRITE_RESP: begin
            S_BVALID = 1'b1;
            S_BRESP = 2'b00;
        end
        READ_ADDR: begin
            S_ARREADY = 1'b1;
            S_RVALID = 1'b1;
            S_RDATA = registers[addr_reg];
            S_RRESP = 2'b00;
        end
        READ_DATA: begin
            S_RVALID = 1'b1;
            S_RDATA = registers[addr_reg];
            S_RRESP = 2'b00;
        end
        default: begin
        end
    endcase
end

endmodule

```

Test Bench:

```
module tb_axi4_lite;

    reg ACLK;
    reg ARESETN;
    reg START_READ;
    reg START_WRITE;
    reg [31:0] address;
    reg [31:0] W_data;
    wire [31:0] R_data;
    wire READ_COMPLETE;
    wire WRITE_COMPLETE;

    wire M_ARREADY;
    wire [31:0] M_ARADDR;
    wire M_ARVALID;
    wire S_ARREADY;
    wire [31:0] S_ARADDR;
    wire S_ARVALID;

    wire [31:0] M_RDATA;
    wire [1:0] M_RRESP;
    wire M_RVALID;
    wire M_RREADY;
    wire [31:0] S_RDATA;
    wire [1:0] S_RRESP;
    wire S_RVALID;
    wire S_RREADY;

    wire M_AWREADY;
    wire [31:0] M_AWADDR;
    wire M_AWVALID;
    wire S_AWREADY;
    wire [31:0] S_AWADDR;
    wire S_AWVALID;

    wire [31:0] M_WDATA;
```

```
wire [3:0] M_WSTRB;  
wire M_WVALID;  
wire M_WREADY;  
wire [31:0] S_WDATA;  
wire [3:0] S_WSTRB;  
wire S_WVALID;  
wire S_WREADY;
```

```
wire [1:0] M_BRESP;  
wire M_BVALID;  
wire M_BREADY;  
wire [1:0] S_BRESP;  
wire S_BVALID;  
wire S_BREADY;
```

```
always #5 ACLK = ~ACLK;
```

```
assign S_ARADDR = M_ARADDR;  
assign S_ARVALID = M_ARVALID;  
assign M_ARREADY = S_ARREADY;  
assign M_RDATA = S_RDATA;  
assign M_RRESP = S_RRESP;  
assign M_RVALID = S_RVALID;  
assign S_RREADY = M_RREADY;  
assign S_AWADDR = M_AWADDR;  
assign S_AWVALID = M_AWVALID;  
assign M_AWREADY = S_AWREADY;  
assign S_WDATA = M_WDATA;  
assign S_WSTRB = M_WSTRB;  
assign S_WVALID = M_WVALID;  
assign M_WREADY = S_WREADY;  
assign M_BRESP = S_BRESP;  
assign M_BVALID = S_BVALID;  
assign S_BREADY = M_BREADY;
```

```
axi4_lite_master #(  
    .ADDR_WIDTH(32),  
    .DATA_WIDTH(32)  
) uut_master (  
    .ACLK(ACLK),  
    .ARESETN(ARESETN),  
    .START_READ(START_READ),
```

```

        .START_WRITE(START_WRITE),
        .address(address),
        .W_data(W_data),
        .R_data(R_data),
        .READ_COMPLETE(READ_COMPLETE),
        .WRITE_COMPLETE(WRITE_COMPLETE),
        .M_ARREADY(M_ARREADY),
        .M_ARADDR(M_ARADDR),
        .M_ARVALID(M_ARVALID),
        .M_RDATA(M_RDATA),
        .M_RRESP(M_RRESP),
        .M_RVALID(M_RVALID),
        .M_RREADY(M_RREADY),
        .M_AWREADY(M_AWREADY),
        .M_AWADDR(M_AWADDR),
        .M_AWVALID(M_AWVALID),
        .M_WDATA(M_WDATA),
        .M_WSTRB(M_WSTRB),
        .M_WVALID(M_WVALID),
        .M_WREADY(M_WREADY),
        .M_BRESP(M_BRESP),
        .M_BVALID(M_BVALID),
        .M_BREADY(M_BREADY)
    );

```

```

axi4_lite_slave #(
    .ADDR_WIDTH(32),
    .DATA_WIDTH(32),
    .NUM_REGISTERS(32)
) uut_slave (
    .ACLK(ACLK),
    .ARESETN(ARESETN),
    .S_ARADDR(S_ARADDR),
    .S_ARVALID(S_ARVALID),
    .S_ARREADY(S_ARREADY),
    .S_RREADY(S_RREADY),
    .S_RDATA(S_RDATA),
    .S_RRESP(S_RRESP),
    .S_RVALID(S_RVALID),
    .S_AWADDR(S_AWADDR),
    .S_AWVALID(S_AWVALID),
    .S_AWREADY(S_AWREADY),

```

```

        .S_WDATA(S_WDATA),
        .S_WSTRB(S_WSTRB),
        .S_WVALID(S_WVALID),
        .S_WREADY(S_WREADY),
        .S_BREADY(S_BREADY),
        .S_BRESP(S_BRESP),
        .S_BVALID(S_BVALID)
    );

initial begin
    ACLK = 0;
    ARESETN = 0;
    START_READ = 0;
    START_WRITE = 0;
    address = 32'h00000000;
    W_data = 32'h00000000;

    #20;
    ARESETN = 1;
    #10;

    $display("Test 1: Write to address 0x00000000");
    address = 32'h00000000;
    W_data = 32'hA5A5A5A5;
    START_WRITE = 1;
    #10;
    START_WRITE = 0;
    wait(WRITE_COMPLETE);
    #10;
    $display("Write Complete at time: %t", $time);

    $display("Test 2: Read from address 0x00000000");
    address = 32'h00000000;
    START_READ = 1;
    #10;
    START_READ = 0;
    wait(READ_COMPLETE);
    #10;
    $display("Read Complete at time: %t, Data = 0x%08h", $time, R_data);

    $display("Test 3: Write to address 0x00000004");
    address = 32'h00000004;

```

```

W_data = 32'h5A5A5A5A;
START_WRITE = 1;
#10;
START_WRITE = 0;
wait(WRITE_COMPLETE);
#10;
$display("Write Complete at time: %t", $time);

$display("Test 4: Read from address 0x00000004");
address = 32'h00000004;
START_READ = 1;
#10;
START_READ = 0;
wait(READ_COMPLETE);
#10;
$display("Read Complete at time: %t, Data = 0x%08h", $time, R_data);

$display("Test 5: Back-to-back operations");
address = 32'h00000008;
W_data = 32'h12345678;
START_WRITE = 1;
#10;
START_WRITE = 0;
wait(WRITE_COMPLETE);

address = 32'h00000008;
START_READ = 1;
#10;
START_READ = 0;
wait(READ_COMPLETE);
$display("Read after Write: Data = 0x%08h", R_data);

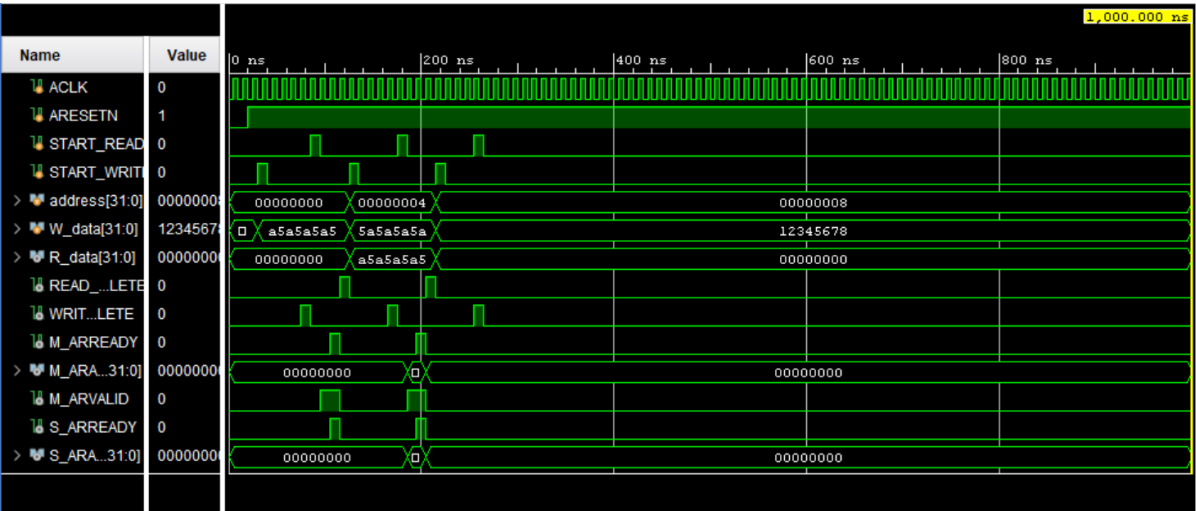
#50;
$display("Simulation completed successfully");
$finish;
end

initial begin
    $dumpfile("waveform.vcd");
    $dumpvars(0, tb_axi4_lite);
end

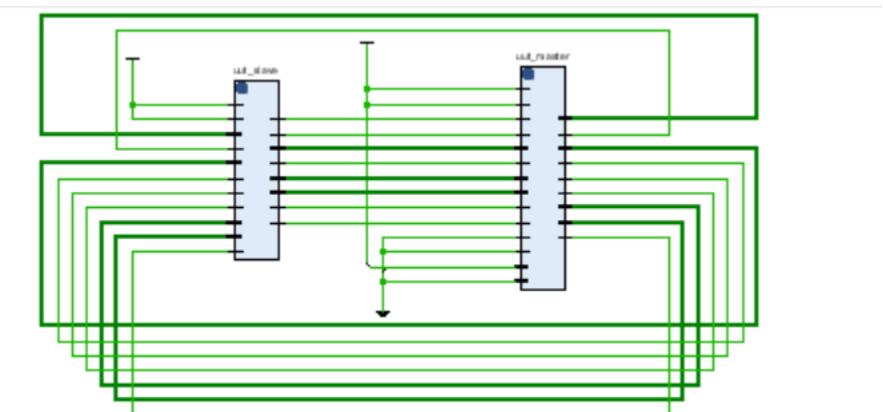
```

endmodule

Wave Form:



Synthesis:



Conclusion:

This design demonstrates the basic operation of an AXI4-Lite slave peripheral. It shows how the five AXI channels work together to perform memory-mapped read and write operations using simple handshake signals. It serves as a good starting point for understanding AXI communication before implementing more advanced features.